DOCUMENTO DE DISEÑO

Fase 1 – Análisis y Diseño
Proyecto FlowDeploy

Proyecto:	FlowDeploy – Pipeline CI/CD automatizado con despliegue continuo
Fase:	Fase 1 – Análisis y Diseño
Equipo:	SN-1
Tutor:	Ing. Carlos G. Rodriguez
Ciclo:	02 - 2026
Fecha de entrega:	18 de octubre de 2026

Integrantes y roles

Integrante	Rol
Herberth Barrera	**Líder**
Juan Francisco Azucena Mejía	**DevOps Jr.**
Diego Alfaro	**Infraestructura Jr.**
Andrew Chicas	**Desarrollador Jr.**
Héctor Benjamín Chipagua Calles	**QA/Analista**

Resumen Ejecutivo

El presente Documento de Diseño define la arquitectura técnica del pipeline de integración y despliegue continuo del proyecto FlowDeploy, tomando como base el documento de análisis. Se describe la arquitectura general: repositorio en GitHub, la automatización mediante GitHub Actions, la aplicación web, el despliegue en la plataforma Render y la verificación mediante un endpoint (/health).
La aplicación base se implementa con Node.js 22, Express, EJS y SQLite. Se estableció una estrategia de ramas basadas en GitHub Flow, con una rama principal protegida y ramas de trabajo por tarea. Se detallan las etapas del CI (validación y pruebas

automatizadas) y del CD (despliegue automático y verificación posterior). Para finalizar se definen las reglas de disparo y aprobación, así como el diagrama del flujo automatizado que garantiza que solo los cambios validados y aprobados lleguen al entorno de producción.

1. Arquitectura general del pipeline

El diseño se organizó en seis capas con responsabilidades bien definidas.

Capa	Tecnología	Función
Control de versiones	GitHub (repositorio público)	Versionar el código, gestionar ramas y disparar la automatización
Automatización	GitHub Actions	Ejecutar los Pipelines de CI y CD definidos en YAML
Ejecución	Runner Ubuntu (efímero)	Correr cada etapa en un entorno limpio y reproducible
Aplicación base	Node.js 22 + Express + EJS + SQLite	Sistema web que es validado y desplegado por el Pipeline
Despliegue	Render (Web Service)	Publicar la aplicación en un entorno accesible
Verificación	Endpoint /health y /version	Confirmar que el despliegue quedó operativo

El flujo es el siguiente: el desarrollador sube cambios a GitHub: GitHub Actions detecta el evento y ejecuta el CI. Si el resultado es correcto y el cambio es aprobado, este se fusiona con main. El merge dispara el CD que invoca un deploy hook de Render y luego valida el endpoint /health

Stack de la aplicación: la aplicación base se implemento con Node.js 22, Express, EJS y SQLite (better-sqlite3). Node 22 es un requisito para el proyecto (campo engines), y SQLite se eligió por simplicidad. La aplicación por medio del desarrollador no ha definido un paso de compilación (build) en su documento técnico, por lo que en este paso el pipeline se limita a instalar dependencias, validar el estilo y ejecutar pruebas

2. Estrategia de ramas y flujo de cambios

Se adoptó el modelo GitHub Flow, este modelo es adecuado para equipos pequeños con entrega continua. Rama principal estable y ramas cortas de trabajo por tarea

Rama	Propósito
Main	Rama principal y protegida. Siempre debe estar desplegada. Solo recibe cambios por Pull Request aprobados
feature/.	Nuevas funcionalidades o componentes
fix/.	Corrección de errores
docs/.	Documentación
ci/.	Cambios en los workflows del Pipeline

Flujo de cambios:

1. Actualizar el main en local.
2. Crear una rama desde main con el prefijo correspondiente.
3. Realizar commits con mensajes significativos.
4. Subir la rama al repositorio.
5. Abrir un Pull Request hacia main.
6. El CI valida automáticamente. El líder del proyecto revisa y aprueba.
7. Fusionar y eliminar la rama.

Reglas:

- No se permite push directo a main.
- Todo cambio entra por Pull Request.
- Se requiere al menos una aprobación.
- El CI debe estar en verde para poder fusionar.
- La rama se elimina después de fusionar.

Justificación:
Se adoptó GitHub Flow porque el equipo es pequeño y se esperan entregas frecuentes. GitHub Flow reduce la sobrecarga de ramas largas y mantiene el main siempre estable, por ello, los activadores (triggers) del CI se limitan a main y a los Pull Request.

3. Etapas del CI y del CD

Integración Continua (CI)

Se ejecuta automáticamente en cada push y en cada Pull Request. Si una etapa falla, el Pipeline se detiene y el resultado es rojo.

#	Etapa	Acción
1	Checkout	Descarga el código del repositorio en el runner
2	Configuración del entorno	Preparar el runtime (Node.js 22) y la caché de dependencias

3	Instalación de dependencias	Ejecutar npm ci para una instalación reproducible
4	Lint	Analizar estilo y errores estáticos con ESLint
5	Pruebas	Ejecutar la suite de pruebas automatizadas

No he colocado etapa de build: el Dev no define un script de compilación, por lo que el CI válida solo con lint y pruebas.

Despliegue Continuo (CD)

Se ejecuta únicamente cuando los cambios llegan a main (merge) o de forma manual.

#	Etapa	Acción
1	Checkout	Descarga el código fusionado
2	Despliegue	Invocar el deploy hook de render mediante una petición POST al secret RENDER_DEPLOY_HOOK_URL. Render construye con npm ci y arranca con npm start (Web Service)
3	Verificación	Consultar el endpoint /health y /version, confirmar respuesta 200 y la versión desplegada, con reintentos por el arranque en frío por desventajas del plan gratuito.

Justificación del deploy hook: Render permite integración directa con GitHub, pero el despliegue lo ejecutaría la plataforma y no el pipeline; con deploy hook el despliegue lo hace GitHub Actions, lo que demuestra el CD dentro del flujo automatizado y mantiene la trazabilidad entre commit y versión publicada.

4. Reglas de aprobación y disparo del despliegue

Evento	Pipeline	Resultado
Push a rama de trabajo	CI	Valida el código (no despliega)
Apertura o actualización de Pull Request	CI	Valida codigo
Pull Request con CI en rojo	No aplica	Merge bloqueado
Pull Request sin aprobación	No aplica	Merge bloqueado
Merge a main	CD	Despliegue automático en Render
Ejecución manual (workflow_dispatch)	CD	Despliegue bajo demanda

Protección de la rama main:

1. Requiere Pull Request antes de fusionar.

2. Requiere al menos una aprobación.
3. Requerir que las validaciones del CI estén en verde.
4. Bloquear el push directo y el force push.

Secrets del repositorio:

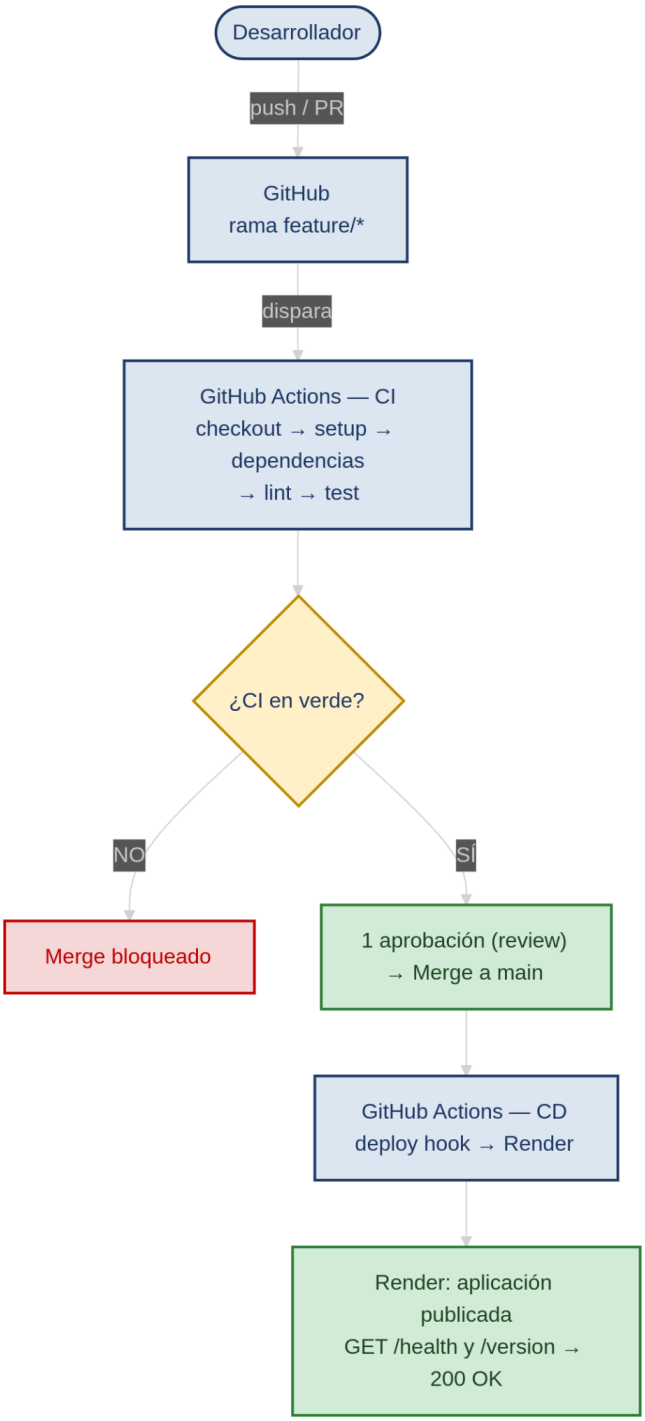
Nombre	Uso
RENDER_DEPLOY_HOOK_URL	Disparar el despliegue en Render desde GitHub Actions
HEALTHCHECK_URL	URL del endpoint de estado para la verificación posterior al despliegue.
APP_VERSION	Versión mostrada por el endpoint /version
GIT_SHA	Commit desplegado, mostrado por /version para trazabilidad

Seguridad:

Ningún token se almacena en el código. Se gestionan exclusivamente como secretos del repositorio y se rotan si se exponen.

5. Diagrama del flujo automatizado

El siguiente diagrama muestra como ningún cambio llega a producción sin pasar por la validación automática y la aprobación humana. La verificación final con /health confirma que el despliegue quedó operando.


Desarrollador
push / PR
GitHub
rama feature/*
dispara
GitHub Actions — CI
checkout → setup → dependencias
→ lint → test
¿CI en verde?
NO
SÍ
Merge bloqueado
1 aprobación (review)
→ Merge a main
GitHub Actions — CD
deploy hook → Render
Render: aplicación publicada
GET /health y /version → 200 OK

6. Conclusión

El diseño define una arquitectura clara y reproducible. GitHub para trazabilidad, GitHub Actions como motor de automatización, Render como plataforma de despliegue y los endpoints /health y /version como verificación final.
La aplicación de una estrategia de ramas como GitHub Flow, junto con la protección de main, garantiza que solo los cambios validados y aprobados lleguen a producción. El uso de deploy hook asegura que el despliegue sea ejecutado por el propio pipeline, así manteniendo la trazabilidad del proceso.

Siguiente paso: En fase 2 se debe ajustar detalles para esclarecer la implementación de Docker, ajustar documentación si se lleva a cabo cambios.
En fase 2 se implementará las etapas reales de CI (lint y pruebas) sobre Node.js 22.
En la fase 3 se implementará el CD con el deploy hook y la verificación de /health y /version

7. Evidencias

- Repositorio: https://github.com/herberth-barrera/FlowDeploy-ESIT
- Pipeline CI: .github/workflows/ci.yml
- Pipeline CD: .github/workflows/cd.yml
- Pull Request #1 (aprobado y fusionado)
- Protección de la rama main evidenciada en el Pull Request.